Part A: Relational Database Design (PostgreSQL)

Task 1: Entity-Relationship (E-R) Diagram

Design Assumptions

1. Each customer must have at least one bank account and may hold multiple bank accounts.
2. Each product belongs to a single type, which is stored as an attribute in the product record without a separate table created for product types.
3. A store can stock multiple products, and a product can be stocked in multiple stores (a many-to-many relationship), resolved by the StoreStock junction table.
4. A single purchase is made by one customer at one store and may include multiple purchase items, with each item corresponding to one product.
5. StockTransfer records the source store, target store, the product being transferred, and the transfer quantity for inter-store stock movements.
6. Identifiers such as customer ID, purchase ID and store ID are auto-generated (SERIAL).
7. All dates must be validated to ensure their validity.

Entity Description

The system contains the following main entities:

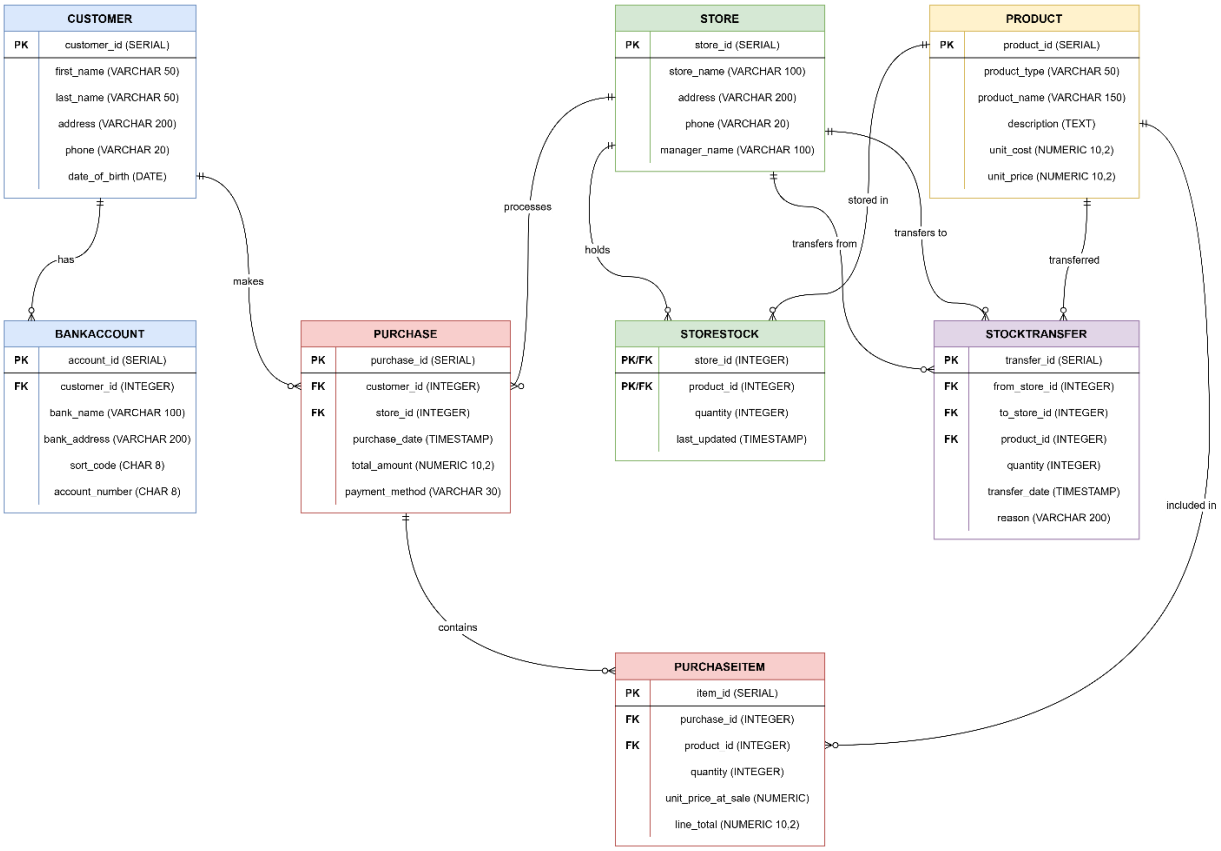
- Customer: Stores basic customer information
- BankAccount: One customer can have multiple bank accounts (1:N relationship)
- Store: Stores store information
- Product: Stores product information
- StoreStock: Resolves the many-to-many relationship between Store and Product
- Purchase: Records a single purchase made by a customer at a store
- PurchaseItem: Resolves the many-to-many relationship between Purchase and Product
- StockTransfer: Records inter-store product transfers

E-R Diagram (Text Description)

The following describes the relationships between entities:

Relationship	Cardinality	Optionality	Description
Customer → BankAccount	1 : N	Mandatory : Optional	One customer can have zero or more bank accounts
Customer → Purchase	1 : N	Mandatory : Optional	One customer can have zero or more purchase records
Store → Purchase	1 : N	Mandatory : Optional	One store can have zero or more purchase records
Purchase → PurchaseItem	1 : N	Mandatory : Mandatory	One purchase must include one or more purchase items
Product → PurchaseItem	1 : N	Mandatory : Optional	One product can appear in zero or more purchase items
Store → StoreStock	1 : N	Mandatory : Optional	One store can have zero or more stock records
Product → StoreStock	1 : N	Mandatory : Optional	One product can be stocked in zero or more stores
Store → StockTransfer (Source)	1 : N	Mandatory : Optional	One store can act as the source for zero or more transfers
Store → StockTransfer (Target)	1 : N	Mandatory : Optional	One store can act as the target for zero or more transfers
Product → StockTransfer	1 : N	Mandatory : Optional	One product can be transferred zero or more times

E-R Diagram (Crow's Foot)



Task 2: Data Dictionary & Normalisation

Table 1: Customer

Attribute	Description	Data Type	Max Size	Constraints
customer_id	Unique customer identifier (auto-generated)	SERIAL	-	PRIMARY KEY, NOT NULL
first_name	Customer's first name	VARCHAR	50	NOT NULL
last_name	Customer's last name	VARCHAR	50	NOT NULL
address_line1	Address line 1	VARCHAR	100	NOT NULL
address_line2	Address line 2	VARCHAR	100	NULL
city	City	VARCHAR	50	NOT NULL
postcode	Postcode	VARCHAR	10	NOT NULL
phone	Phone number	VARCHAR	20	NOT NULL, Format: numbers/+/spaces

date_of_birth	Date of birth	DATE	-	NOT NULL, CHECK(date_of_birth < CURRENT_DATE)
created_at	Record creation time	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Functional Dependencies:

customer_id → first_name, last_name, address_line1, address_line2, city, postcode, phone, date_of_birth, created_at

3NF Justification: The primary key customer_id determines all other attributes. There are no partial dependencies (no composite primary key) or transitive dependencies (all non-key attributes depend directly on the primary key), so the table satisfies the Third Normal Form.

Table 2: BankAccount

Attribute	Description	Data Type	Max Size	Constraints
account_id	Unique bank account identifier	SERIAL	-	PRIMARY KEY, NOT NULL
customer_id	Associated customer ID	INTEGER	-	NOT NULL, FK → Customer(customer_id)
bank_name	Bank name	VARCHAR	100	NOT NULL
bank_address	Bank address	VARCHAR	200	NOT NULL
sort_code	Bank sort code (Format: XX-XX-XX)	CHAR	8	NOT NULL, CHECK(sort_code ~ '^[0-9]{2}-[0-9]{2}-[0-9]{2}$')
account_number	Bank account number (8 digits)	CHAR	8	NOT NULL, CHECK(account_number ~ '^[0-9]{8}$'), UNIQUE(sort_code, account_number)

Functional Dependencies:

account_id → customer_id, bank_name, bank_address, sort_code, account_number

3NF Justification: The primary key account_id determines all other attributes. There are no partial dependencies or transitive dependencies, so the table satisfies the Third Normal Form.

Table 3: Store

Attribute	Description	Data Type	Max Size	Constraints
store_id	Unique store identifier	SERIAL	-	PRIMARY KEY, NOT NULL
store_name	Store name	VARCHAR	100	NOT NULL, UNIQUE
address_line1	Store address line 1	VARCHAR	100	NOT NULL
address_line2	Store address line 2	VARCHAR	100	NULL
city	City	VARCHAR	50	NOT NULL
postcode	Postcode	VARCHAR	10	NOT NULL
phone	Store phone number	VARCHAR	20	NOT NULL
manager_name	Store manager's name	VARCHAR	100	NULL

Functional Dependencies:

store_id → store_name, address_line1, address_line2, city, postcode, phone, manager_name

3NF Justification: The primary key store_id determines all other attributes. There are no partial dependencies or transitive dependencies, so the table satisfies the Third Normal Form.

Table 4: Product

Attribute	Description	Data Type	Max Size	Constraints
product_id	Unique product	SERIAL	-	PRIMARY KEY, NOT NULL

Attribute	Description	Data Type	Max Size	Constraints
	identifier			
product_type	Product type (e.g., toy/computer game)	VARCHAR	50	NOT NULL
product_name	Product name	VARCHAR	150	NOT NULL
description	Product description	TEXT	-	NULL
unit_cost	Unit cost (GBP)	NUMERIC	10,2	NOT NULL, CHECK(unit_cost > 0)
unit_price	Retail unit price (GBP)	NUMERIC	10,2	NOT NULL, CHECK(unit_price > 0)
created_at	Product entry time	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Functional Dependencies:

product_id → product_type, product_name, description, unit_cost, unit_price, created_at

3NF Justification: The primary key product_id determines all other attributes. There are no partial dependencies or transitive dependencies, so the table satisfies the Third Normal Form.

Table 5: StoreStock

Attribute	Description	Data Type	Max Size	Constraints
store_id	Store ID (composite primary key)	INTEGER	-	NOT NULL, FK → Store(store_id), PK(store_id, product_id)
product_id	Product ID (composite primary key)	INTEGER	-	NOT NULL, FK → Product(product_id), PK(store_id, product_id)

Attribute	Description	Data Type	Max Size	Constraints
quantity	Current stock quantity	INTEGER	-	NOT NULL, DEFAULT 0, CHECK(quantity >= 0)
last_updated	Last update time	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Functional Dependencies:

(store_id, product_id) → quantity, last_updated

3NF Justification: The composite primary key (store_id, product_id) determines all non-key attributes. There are no partial dependencies or transitive dependencies, so the table satisfies the Third Normal Form.

Table 6: Purchase

Attribute	Description	Data Type	Max Size	Constraints
purchase_id	Unique purchase record identifier	SERIAL	-	PRIMARY KEY, NOT NULL
customer_id	Purchasing customer ID	INTEGER	-	NOT NULL, FK → Customer(customer_id)
store_id	Purchasing store ID	INTEGER	-	NOT NULL, FK → Store(store_id)
purchase_date	Purchase date and time	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
total_amount	Total purchase amount	NUMERIC	10,2	NOT NULL, CHECK(total_amount >= 0)
payment_method	Payment method	VARCHAR	30	NOT NULL, CHECK(payment_method IN ('cash','card','online'))

Functional Dependencies:

purchase_id → customer_id, store_id, purchase_date, total_amount, payment_method

3NF Justification: The primary key purchase_id determines all other attributes. There are no partial dependencies or transitive dependencies, so the table satisfies the Third Normal Form.

Table 7: PurchaseItem

Attribute	Description	Data Type	Max Size	Constraints
item_id	Unique purchase item identifier	SERIAL	-	PRIMARY KEY, NOT NULL
purchase_id	Associated purchase record ID	INTEGER	-	NOT NULL, FK → Purchase(purchase_id)
product_id	Associated product ID	INTEGER	-	NOT NULL, FK → Product(product_id)
quantity	Purchase quantity	INTEGER	-	NOT NULL, CHECK(quantity > 0)
unit_price_at_sale	Unit price at the time of purchase (snapshot)	NUMERIC	10,2	NOT NULL, CHECK(unit_price_at_sale > 0)
line_total	Line total for the item	NUMERIC	10,2	NOT NULL, CHECK(line_total > 0)

Functional Dependencies:

item_id → purchase_id, product_id, quantity, unit_price_at_sale, line_total

3NF Justification: line_total is calculated as quantity × unit_price_at_sale but stored independently as a snapshot to ensure the accuracy of historical records. No partial or transitive dependencies exist, so the table satisfies the Third Normal Form.

Table 8: StockTransfer

Attribute	Description	Data Type	Max Size	Constraints
transfer_id	Unique stock transfer record identifier	SERIAL	-	PRIMARY KEY, NOT NULL
from_store_id	Source store ID	INTEGER	-	NOT NULL, FK → Store(store_id)
to_store_id	Target store ID	INTEGER	-	NOT NULL, FK → Store(store_id), CHECK(from_store_id <> to_store_id)
product_id	Transferred product ID	INTEGER	-	NOT NULL, FK → Product(product_id)
quantity	Transfer quantity	INTEGER	-	NOT NULL, CHECK(quantity > 0)
transfer_date	Transfer date and time	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
reason	Transfer reason/remarks	VARCHAR	200	NULL

Functional Dependencies:

transfer_id → from_store_id, to_store_id, product_id, quantity, transfer_date, reason

3NF Justification: The primary key transfer_id determines all other attributes. There are no partial dependencies or transitive dependencies, so the table satisfies the Third Normal Form.

Part C: Critical Discussion

Utilisation of MongoDB Document Store Features and DBMS Selection Analysis

When converting TTE's relational database to a MongoDB document store, the core NoSQL features leveraged are as follows:

First, the number of collections is significantly reduced through nested documents. In the relational design, customer data is split across two separate tables (Customer and

BankAccount); in MongoDB, bank account information is directly nested as an array within each customer document:

{ "_id": ObjectId(...), "first_name": "John", "bank_accounts": [{ "bank_name": "Barclays", "sort_code": "20-00-00", "account_number": "12345678" }] }

Second, purchase items (PurchaseItem) in purchase records are nested as an array within the Purchase document, eliminating the need for JOIN operations in the relational database. All items for a single purchase can be retrieved with one query, improving read performance.

Through this design, the original 8 relational tables are consolidated into 4 MongoDB collections (customers, stores, products, purchases), a reduction of approximately 50% in the number of collections.

Recommendation for DBMS Selection: A relational database (PostgreSQL) is the more suitable choice for TTE, for the following reasons:

1. TTE's data is highly structured, with clear relationships between products, customers and sales—this is the core strength of relational databases.
2. The ACID transaction guarantees of relational databases ensure data consistency for sensitive operations such as sales record keeping and inventory updates.
3. TTE requires complex management reports; SQL has mature optimisation mechanisms for such multi-table aggregate queries, whereas MongoDB's aggregation pipeline, while powerful, is less intuitive for complex relational queries. NoSQL is better suited for scenarios with unstructured data schemas and requirements for horizontal scaling—needs that TTE does not fully have at this stage.